KUTAISI
INTERNATIONAL
UNIVERSITY

LEPL Kutaisi International University
School of Computer Science
Bachelor Program of Computer Science

Mirian Shilakadze
Mikheili Kvinikadze
Konstantine Machavariani

C0 to Machine Code: Assembler, Simulator and Basic OS Support.

Thesis submitted for the degree of Bachelor of Computer Science

Bachelor's Thesis Supervisor: Prof. Dr. Walter Tichy

Co-supervisor: Prof. Dr. Wolfgang J. Paul

Kutaisi, 2026

Declaration

We, the authors of this bachelor's thesis, declare that the thesis is our own original work and does not contain any material previously published, submitted for publication, or presented for defense by other authors, unless properly referenced or cited in accordance with academic standards

Mirian Shilakadze Date: 29.06.2026

Mikheili Kvinikadze Date: 29.06.2026

Konstantine Machavariani Date: 29.06.2026

Abstract

This thesis presents the development of a complete toolchain that takes the C0 programming language code and compiles it into a custom bytecode format. Alongside this, we built a minimal operating system kernel to execute the resulting programs on an FPGA board.

We started from an existing C0-to-MIPS compiler and extended it in several ways. First, we wrote standalone assembler that converts MIPS instructions into bytecode. We built a MIPS simulator to help with debugging. We extended the C0 grammar into a "C0 + Assembly" dialect, that adds inline assembly and direct register access to standard C0. These features are necessary for writing operating systems code. Extended language was kept within $LR(1)$ language class.

Furthermore, we achieved our major goal of implementing a basic OS-kernel. We implemented a *bootloader* that reads translated kernel code from disk and loads it into memory. We wrote *runvm* primitive capable of running virtual user's code and deals with page fault interrupts.

The end result result is a full pipeline. Code written "C0 + Assembly" goes through compiler then the assembler, gets loaded by *bootloader* and runs under kernel on target hardware.

Keywords: C0, compiler, MIPS, assembler, bytecode, simulator, bootloader, virtual machine, page fault, LR(1) grammar, FPGA, instruction set architecture.

Contents

Declaration I
Abstract II
1. Introduction 1
   1.1. Overview 1
   1.2. Author Contributions 1
2. MIPS ISA Overview 2
   2.1. MIPS configuration and Instruction Fields 4
3. Assembler 5
   3.1. Assembler Input 5
   3.2. Assembler Implementation 6
      3.2.1. Labels 6
      3.2.2. Macros 7
4. Extending C0 Grammar and Compiler 8
   4.1. Overview 8
   4.2. C0 + Assembly 8
      4.2.1. Grammar 8
      4.2.2. DK_1 Test 10
      4.2.3. Implementation 10
   4.3. Void Function 11
      4.3.1. Grammar 11
      4.3.2. Derivation 12
      4.3.3. Implementation 12
   4.4. Examples 13
   4.5. Corrections to Pre-Existing Bugs 14
      4.5.1. Macros Inside Conditional Statement 14
      4.5.2. Constant Evaluation 15
      4.5.3. Array Element Generation 15
5. Simulator 16
   5.1. Architectural State Model 16
      5.1.1. Configuration 16
      5.1.2. Paged Memory 17
      5.1.3. Memory Access and Byte Ordering 17
   5.2. Simulation Cycle 18
   5.3. Instruction Registry 19
   5.4. C0-Programs Simulation 19
   5.5. Translation-to-Simulation: Challenges & Solutions 20
      5.5.1. Initialize Stack Pointers 20
      5.5.2. Function call 21
   5.6. Verification 22
6. Basic OS-Support 23
   6.1. Overview 23
   6.2. Basic Kernel: Technical Aspects 24
   6.3. *Main Memory System* Layout 25
   6.4. Kernel Data Structures & Initialization 25
      6.4.1. Address Translation Support 25
      6.4.2. Process Control Block 26
   6.5. Simple Scheduling Mechanism 26

6.6. Kernel Primitive – *runvm* 27
6.7. Bootloader 29
6.7.1. Interrupt Signal Inspection 29
6.7.2. Fetching Kernel Code 29
6.7.3. Adjusting Jump Lengths 30
7. Conclusion: Limitations & Future Work 31
7.1. Overview 31
7.2. Limitations 31
7.3. Future Work 31
Bibliography 32
Appendix 33
A.1 C0 Grammar 33
A.2 Instruction Set Implementation 34
A.2.1 The Instruction Registry 36
A.3 End-to-End Simulation 37
A.3.1 Step 1: C0 to MIPS Assembly 39
A.3.2 Step 2: MIPS Assembly to Bytecode 40
A.3.3 Step 3: Simulation and Final State 40
A.4 Program Tests 41
A.5 Kernel C0-Program 42
A.6 Bootloader: Kernel Fetch Routine 43
A.7 ROM Initialization 43

1. Introduction

1.1. Overview

Thesis covers two things: a toolchain that compiles C0 code down to bytecode and a basic operating system kernel that runs the resulting programs on target FPGA board. The C0 language borrows C's syntax but follows the semantics of FreePascal. The language is formally defined by a context-free grammar, as detailed in System Architecture: An Ordinary Engineering Discipline [1]. This grammar is illustrated in Appendix Figure 11.

Our work began with an existing C0 compiler and parser project developed at Kutaisi International University, by Toma Pirtskhelani and Irakli Khotchava [2], which could take C0 source code and produce MIPS assembly. We utilized its core components: lexical analysis, parsing, and initial code generation and used them as a base for our extensions.

One of the first things we added was an assembler that takes MIPS instructions and converts them into byte format our target machine expects.

Writing an OS kernel in plain C0 turned out to be impossible. The language had no way to touch registers directly or drop into raw assembly when needed. So we extended the grammar and parser to support both. The result is what we call "C0 + Assembly", which adds inline assembly and direct register access to the language. We also added support for void function declarations. The extended language still parses as $LR(1)$ language., which we verified.

As the project evolved into a multi-layered system, testing and debugging became a significant difficulty. To facilitate this process, we implemented a basic MIPS simulator. This tool proved invaluable for testing the translation process from C0 to bytecode and for identifying bugs in the assembly generation phase. It is important to note that the simulator's purpose is to aid in debugging the translation pipeline, not to formally verify the correctness of the generated code.

Finally, to achieve the ultimate goal – simulating several *users'* programs on FPGA board – we've implemented core OS services that are capable of being simulated on the hardware supporting - *Disk accesses, Interrupts and MMU*. At present, we are unable to test these services on the actual board, as the corresponding hardware designs were not finalized early enough. Our implementation focuses on two critical components: a *bootloader*, which handles the correct detection of a *reset* signal and loads the translated kernel code from disk, and a virtual machine runner (*runvm*), which manages virtual users and provides transparent handling of invalid-page-fault interrupts. See Section A.5.

The remainder of this thesis details the components of our system. Chapter 3 addresses the MIPS-to-bytecode assembler. Chapter 4 steps back to describe the critical modifications to the C0 compiler, which resulted in the "C0 + Assembly" dialect used for kernel development. To support this multi-stage translation, Chapter 5 introduces the MIPS simulator built for debugging and validation. Finally, Chapter 6 presents the design and implementation of the operating system kernel, focusing on the *bootloader* and *runvm* component that brings all previous parts together.

The entire implementation of project, written in Java, is accesible on GitHub.

1.2. Author Contributions

The development of this project was a collaborative effort, with primary responsibilities for the core components and corresponding thesis chapters distributed as follows:

- Chapter 3 focusing on the MIPS assembler is done by Mirian Shilakadze.
- Chapter 4 and Chapter 6 focusing on compiler modifications and kernel is done by Mikheili Kvinikadze.
- Chapter 5 focusing on instruction simulation is done by Konstantine Machavariani.

2. MIPS ISA Overview

This chapter provides concise overview of the MIPS ISA, closely following the formal definitions presented in [1]. The instruction set and assembler syntax is presented in tables (Table 1, Table 2, Table 3), followed by precise interpretation of the tables in Chapter 2.1.

opc	rt	Mnemonic	Assembler Syntax	Effect
Data Transfer				
100011		lw	lw rt rs imm	rt = m
101011		sw	sw rt rs imm	m = rt
Arithmetic, Logic, Test-and-Set				
001000		addi	addi rt rs imm	rt = rs + sxt(imm)
001001		addiu	addiu rt rs imm	rt = rs + sxt(imm)
001010		slti	slti rt rs imm	rt = (rs < sxt(imm) ? 1_{32} : 0_{32})
001011		sltiu	sltiu rt rs imm	rt = (rs < sxt(imm) ? 1_{32} : 0_{32})
001100		andi	andi rt rs imm	rt = rs $\wedge$ zxt(imm)
001101		ori	ori rt rs imm	rt = rs $\vee$ zxt(imm)
001110		xori	xori rt rs imm	rt = rs $\oplus$ zxt(imm)
001111		lui	lui rt imm	rt = imm2^{16}
Branch				
000001	00000	bltz	bltz rs imm	pc = pc + (rs < 0 ? imm00 : 4_{32})
000001	00001	bgez	bgez rs imm	pc = pc + (rs $\geq$ 0 ? imm00 : 4_{32})
000100		beq	beq rs rt imm	pc = pc + (rs = rt ? imm00 : 4_{32})
000101		bne	bne rs rt imm	pc = pc + (rs $\neq$ rt ? imm00 : 4_{32})
000110	00000	blez	blez rs imm	pc = pc + (rs $\leq$ 0 ? imm00 : 4_{32})
000111	00000	bgtz	bgtz rs imm	pc = pc + (rs $\geq$ 0 ? imm00 : 4_{32})

Table 1: **I-type** instructions

opcode	fun	rs	Mnemonic	Assembler Syntax	Effect
Shift Operation					
000000	000010		srl	srl rd rt sa	rd = srl(rt, sa)
Arithmetic, Logical Operation					
000000	100000		add	add rd rs rt	rd = rs + rt
000000	100001		addu	addu rd rs rt	rd = rs + rt
000000	100010		sub	sub rd rs rt	rd = rs - rt
000000	100011		subu	subu rd rs rt	rd = rs - rt
000000	100100		and	and rd rs rt	rd = rs $\wedge$ rt
000000	100101		or	or rd rs rt	rd = rs $\vee$ rt
000000	100110		xor	xor rd rs rt	rd = rs $\oplus$ rt
000000	100111		nor	nor rd rs rt	rd = rs $\vee$ $\bar{rt}$
Test-and-Set Operation					
000000	101010		slt	slt rd rs rt	rd = (rs < rt ? 1_{32} : 0_{32})
000000	101011		sltu	sltu rd rs rt	rd = (rs < rt ? 1_{32} : 0_{32})
Jumps, System Call					
000000	001000		jr	jr rs	pc = rs
000000	001001		jalr	jalr rd rs	rd = pc + 4_{32} pc = rs
000000	001100		sysc	sysc	System Call
Coprocessor Instructions					
010000	011000	10000	eret	eret	Exception Return
010000		00100	movg2s	movg2s rd rt	spr[rd] := gpr[rt]
010000		00000	movs2g	movs2g rd rt	gpr[rt] := spr[rd]

Table 2: **R-type** instructions

opc	Mnemonic	Assembler Syntax	Effect
Jumps			
000010	j	j iindex	pc = pc + sxt(iindex)
000011	jal	jal iindex	R31 = pc + 4_{32}, pc = pc + sxt(iindex)

Table 3: **J-type** instructions

2.1. MIPS configuration and Instruction Fields

A MIPS configuration c has 3 data structures:

- $c.pc \in \mathbb{B}^{32}$: The program counter; points to the instruction to be executed.
- $c.gpr : \mathbb{B}^5 \to \mathbb{B}^{32}$: 32 general-purpose registers each 32 bit wide. For register address $x \in \mathbb{B}^{32}$, $c.gpr(x) \in \mathbb{B}^{32}$ denotes the content of register x in configuration c. Register 0_5 is always 0_{32}.

$$c.gpr(0_5) = 0_{32}$$

- $c.m : \mathbb{B}^{32} \to \mathbb{B}^8$: The byte-addressable processor memory. For address $a \in \mathbb{B}^{32}$, $c.m(a) \in \mathbb{B}^8$ defines the content of the memory location a in configuration c.

For memory $m : \mathbb{B}^{32} \to \mathbb{B}^5$, address $a \in \mathbb{B}^{32}$ and a natural number $d \in \mathbb{N}$, we denote the content of d consecutive bytes by

$$m_1(a) = m(a)$$
$$m_{d+1}(a) = m(a +_{32} d_{32}) \circ m_d(a)$$

where $\circ : \mathbb{B}^n \times \mathbb{B}^m \to \mathbb{B}^{n+m}$ denotes concatenation operator. $I(c)$ defines the current instruction to be executed in configuration c:

$$I(c) = c.m_4(c.pc)$$

The six high-order bits of the instruction are called the opcode of the instruction:

$$opc(c) = I(c)[31 : 26]$$

There are 3 types of MIPS instructions: **I-type**, **R-type** and **J-type**. Depending on the instruction type, the bits of an instruction is subdivided sections as shown in Figure 1. The instruction type is determined by the following predicates:

$$rtype(c) \equiv opc(c) = 0^*0^4$$
$$jtype(c) \equiv opc(c) = 0^41^*$$
$$itype(c) \equiv \overline{rtype(c) \vee jtype(c)}$$

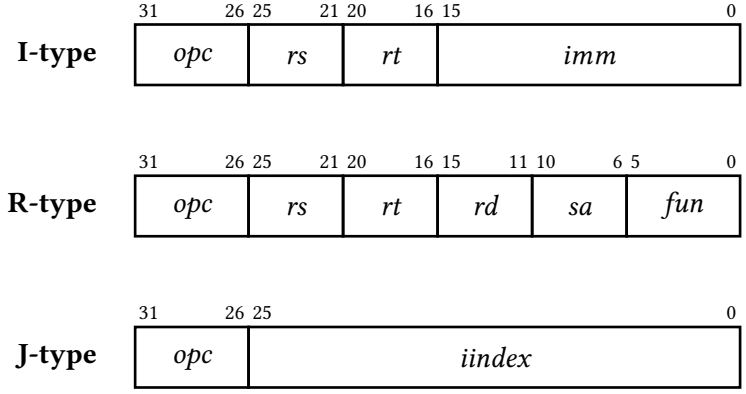


Figure 1: Types and fields of MIPS instructions

Addresses of registers in the general-purpose register file are specified in the following fields of the instruction:

$$rc(c) \equiv I(c)[25 : 21]$$
$$rt(c) \equiv I(c)[20 : 16]$$
$$rd(c) \equiv I(c)[15 : 11]$$

Two kinds of immediate constants can be specified:

$$imm(c) \equiv I(c)[15 : 0]$$
$$iindex(c) \equiv I(c)[25 : 0]$$

In order to apply operations to them, the constant can be extended with 16 high-order bits in two ways: zero extension and sign extension:

$$zximm(c) \equiv 0^{16} \circ imm(c)$$
$$sximm(c) \equiv imm(c)[15]^{16} \circ imm(c)$$

3. Assembler

This chapter presents the implementation of the MIPS assembler. Our objective was to develop an assembler capable of translating assembly code to binary in a single pass, while minimizing memory usage. To achieve this, we aimed to emit the binary representation of each instruction to the output file as early as possible, thereby avoiding the need to store intermediate results in memory.

3.1. Assembler Input

The input to our assembler is a file (or stream) containing an assembly program written in our custom syntax. An example of such program is shown in Code 1.

```
macro: gpr(30) = enc(10240, uint)
macro: gpr(28) = enc(5120, uint)
addiu 29 28 16
subu 1 29 28
srl 1 1 2
macro: zero(28, 1)
macro: gpr(28) = enc(5120, uint)
j _main

_main:
addi 1 28 0
macro: gpr(2) = enc(2, int)
sw 2 1 0
addi 1 28 4
macro: gpr(2) = enc(10, int)
sw 2 1 0
addi 1 29 0
addi 1 1 -9216
blez 1 4
macro: gpr(1) = enc(41, uint)
sysc
addi 29 29 4
jal _swap
macro: gpr(1) = enc(1, int)
sysc

_swap:
sw 31 29 -4
addi 1 28 0
lw 1 1 0
addi 29 29 -4
jr 1
```

Code 1: Example of an assembler input

Each non-blank line in the source is one of three forms:

1. **Instruction**: as defined in *Assembler Syntax* section of Table 1, Table 2, Table 3 with the addition of **J-type** instructions that can take labels as operands (e.g. `j _main`), in which case the jump target is the address marked by that label.

2. **Macro invocation**: use of a predefined pseudo-operation (e.g. `macro: gpr(28) = enc(5120, uint)`), expanded inline to one or more real instructions.
3. **Label definition**: an identifier given by regular expression `$_[A-Za-z]:^` (e.g. `_main:` or `_swap:`) that marks the address of the very next instruction or macro invocation.

3.2. Assembler Implementation

The assembler processes the input one line at a time, emitting the corresponding bytecode as early as possible. As described previously, each non-blank line in the input falls into one of three categories, and the assembler handles each form differently:

1. **Instructions**: For this form, the bytecode can typically be emitted immediately. The only exceptions involve interactions with label definitions and references, which are discussed in Chapter 3.2.1.
   To support efficient opcode lookup, operand parsing, and argument ordering, without resorting to large `switch` statements, we use an `InstructionRegistry` implemented using the factory pattern. This design allows each instruction format to be registered declaratively. When the assembler encounters an instruction, it queries the registry to retrieve the corresponding encoder, syntax, and parsing rules. Implementation are provided in the appendix (Section A.2).
2. **Macro Invocation**: Each macro invocation is expanded into one or more internal instruction objects (`Instruction` class shown in Code 26), which are then handled exactly like ordinary instructions, including any necessary delays for label resolution, before bytecode emission. More on them in Chapter 3.2.2.
3. **Label definition**: When the assembler encounters a label definition, its current bytecode offset is recorded to be used for future **J-type** instructions. If the label was previously used, all pending instructions waiting on it are now resolved and emitted. Further details on labels are given in Chapter 3.2.1.

3.2.1. Labels

A key challenge in assembler implementation of labels is handling *forward references*, where a label is used as an operand before its definition. In such cases, neither the machine code for that instruction nor for any subsequent instructions can be emitted until the label's address has been resolved. Consider the assembly code in Code 2.

```
1 j _loop       # forward-refers to _loop
2 addi 2 2 1    # also deferred until _loop is known
3
4 _loop:
5 subu 3 3 3
6 jr 31
```

Code 2: Example of a label forward reference

Since the assembler processes input in a single, line-by-line pass, it cannot emit the machine code for `j _loop` as `_loop` is not defined yet. We call any such instruction *unresolved*. Since output machine code is streamed in order, the very next instruction `addi 2 2 1` must also be deferred until its predecessor is resolved. This chain of deferral continues for every instruction that follows, up to the point where the assembler finally encounters `_loop:`, at that moment, it records the label's byte-offset, resolved jump and emits all the pending instructions.

For labels that are defined before their use, however, any jump referring to them is marked resolved immediately and its machine code is emitted on the spot.

To implement this deferral logic, we introduced an instruction queue in the assembler. Every instruction (whether coming directly from source or expanded from a macro) is enqueued before any emission is

attempted. After processing each input line, we invoke a drain routine that walks the queue, emitting all instructions whose labels have now been resolved. In pseudocode, the drain function is given in Code 3.

```
FUNCTION drainInstructions():
  // 1. Attempt to emit as many resolved instructions as possible
  WHILE queue IS NOT EMPTY:
    instr = queue.PEEK()
    // 2. If the next instruction is still waiting on a label, stop
    IF NOT instr.isResolved():
      BREAK
    END IF
    // 3. Remove and emit the resolved instruction
    queue.POP()
    emit(instr)
  END WHILE

// 4. Any remaining instructions in the queue are still unresolved
END FUNCTION
```

Code 3: Instruction queue drain function

3.2.2. Macros

Macros provide reusable MIPS code snippets which can be accessed as a single instruction anywhere in assembly. It is generally exploited to encapsulate repetitive tasks such as — software division, multiplication, constant evaluation, etc. At translation time, each macro is expanded into its underlying instruction set, keeping the source assembly code concise and readable, while generating the full instruction sequence in the bytecode.

Below, we define a few key macros, their specification details are covered in Chapter 9.1 [1]:

- `gpr(i) = enc(val, simple type)` – used to evaluate constant values.
- `zero(i, j)` – initializes $gpr(j)$ memory words starting from address $gpr(i)$. After completing this macro both registers - i and j are modified.
- `mul(k, i, j)` – encodes software multiplication of binary numbers specified by registers i and j and stores result in register k. Note that both signed and unsigned integer multiplication are supported by this macro.
- `divu(k, i, j)` – provides unsigned integer division of binary numbers specified by registers i and j. Consequently, result is stored in register k.
- `divt(k, i, j)` – executes division of signed integer numbers, specifications follow $divu(k, i, j)$.

Later for kernel programming, this list is extended with four new instructions.

- `ssave(r)` – stores register i into its corresponding address in scratch memory (see Chapter 15.5. [1]).
- `srestore(r)` – restores register i from scratch memory (see Chapter 15.5.1 [1]).
- `restore-user` – it restores current user process and its configuration, while saving host's portions accordingly (see Chapter 15.5.4 [1]).
- `save-user` – Upon a failed *reset* signal check, this macro saves current user process and restores host's configuration. (see Chapter 15.5.7 [1])

All these macro-expansion logic is included inside class – `Macros.java`, which defines for each pseudo-instruction respective expansion into the underlying MIPS sequence.

```
macro: gpr(1) = enc(-1, int)
```

```
# lui 1 (upper half of -1)
00111100000000001111111111111111
# ori 1 1 (lower half of -1)
00110100001000011111111111111111
```

Code 4: Macro expansion

4. Extending C0 Grammar and Compiler

4.1. Overview

Kernel programming often requires accessing data outside the scope of standard C0 variables. In this case, data can come from:

1. registers and their content.
2. memory addresses outside the range of C0 variables.

To handle (1) one should extend the grammar with instructions that enable direct register access for reading and writing C0 variables (see Chapter 13.3 [1]).

$$gpr(i) = id \quad \{J\}$$

or,

$$id = gpr(i) \quad \{J\}$$

Here, J specifies the set of registers that should remain unchanged for the statement translation, it can be empty as well. Note that when reading from register i, we also don't use it throughout id evaluation, despite not being directly mentioned in J.

For (2), we introduce plain inline assembly on C0 level. Besides standard memory accesses with *lw* and *sw*, all other MIPS instructions can be used while programming on high-level.

$$asm(\text{MIPS})$$

Later for the kernel implementation, we provide the primitive functions such as *copyms*, *readdisk*, *writedisk*, etc. As all these primitives don't indent to return any value, one intuitively observes the need of void functions in C0. During code generation, void functions are translated similarly to regular functions (see Chapter 12.4 [1]).

$$void \quad f(t_1 \ x_1, ..., t_p \ x_p) \quad \{t_{p+1} \ x_{p+1}, ..., t_r \ x_r; \ body\}$$

Major challenge in extending grammar is to keep the resulting language in $LR(1)$ class. After constructing automaton, this is normally validated with DK_1 test. For full details see Chapter 2.4 [3].

Because our work highly hinges on the existing compiler and parser [2], we leave out the unchanged code generation details and instead exclusively describe the extension points and the implementation of our modifications.

This chapter ends with the report on those bugs we detected so far during translation phase. For every issue, we explain the problem and outline the changes made to resolve it.

4.2. C0 + Assembly

4.2.1. Grammar

First, we discuss how original grammar - Figure 11 was adapted, and what other considerations are taken to fulfill DK_1 test. To generate set of registers J, the grammar is extended with nonterminal – Number Set <NuS> symbol.

```
<Di>  -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<DiS> -> <Di> | <Di><DiS>
<NuS> -> <DiS> | <DiS>,<NuS>

  .
  .
  .

<St> -> ... | <id>=gpr(<DiS>) | <id>=gpr(<DiS>) {<NuS>} |
     -> ... | gpr(<DiS>)=<id> | gpr(<DiS>)=<id> {<NuS>}
```

Code 5: Reading and writing to registers

As it is depicted on Code 5, <NuS> generates the sequence of <DiS>. To specify the register i – we simply use <DiS> node. Additionally, "*gpr*" is required to be included in the terminal symbols list. Finally, we finish extensions to <St> with statements reading and writing from *gpr*(<DiS>).

```
<Di>      -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<DiS>     -> <Di> | <Di><DiS>
<Le>      -> A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | ...

<DiLe>    -> <Le> | <Di>
<DiLeS>   -> <DiLe> | <DiLe><DiLeS>

<DiLeCC>  -> - | <DiLe>
<DiLeCCS> -> <DiLeCC> | <DiLeCCS><DiLeCC>
<ASM>     -> <DiLeCCS> | <ASM> <DiLeCCS>

<St>      -> ... | asm( <ASM> )
```

Code 6: Inline assembly

For inline assembly instructions, Code 6 shows the presence of three additional nonterminal symbols.

- <DiLeCC> – extends digit/letter with "-". The last alteration suffices describing negative intermediate values for *lw*, *sw* (e.g. `sw  31  29  -(size($f)+8)` first statement of <body> translation).
- <DiLeCCS> – simply generates sequence of <DiLeCC>.
- <ASM> – combines arbitrary number of words containing $Letters \cup Digits \cup \{\text{-}\}$ separated with " ".

Apart from MIPS, adapted grammar may also cover almost every random sequence of digits/letters. We would not stress ourselves with this exception, as assembly instructions are usually referred specifically only for kernel programming. Finally, complete the extension of <St> similarly with asm(<ASM>) and include "asm" symbol in the terminals list.
Note that current grammar generates each MIPS instruction separately (see Code 7).

```
int readms(uint a){ // software read from main memory

    int tmp;
    gpr(1) = a;
    asm( lw 2 1 0 );
    asm( sw 2 29 -4 );
    return tmp
}
```

Code 7: Primitive - *readms*

4.2.2. DK_1 Test

We didn't make any changes to parser generation [2] and therefore, DK_1 automaton is obtained as it was for the original grammar. Alongside this, we expect to encounter similar problems mentioned in Chapter 2.6 [2] that could fail DK_1 test.

Unfortunately, our initial attempt of introducing MIPS was unsuccessful.

```
R1:     <ASM>  ->  <DiLeCCS>
R2:     <ASM>  ->  <DiLeCCS>. <ASM>
```

In both case follow-up look-ahead symbol is – " ", the latter makes *accept* state failing (2) condition of DK_1 test. To prevent this, we also invert the derivation order as it is shown at Code 6.

4.2.3. Implementation

Our overall idea was to keep code generation process almost the same to use most of available portions as black boxes. Therefore, all these changes follow the same intention. Inside class `CodeGenerator.java` while iterating through derivation tree, we modify function `generateStS`. Now, it additionally checks existence of both reading and writing to registers and inline assembly. This inspection purely depends on the border word structure of `DTE st`, and once it is observed, one of the following functions are called: `generateReadGPR, generateWriteGPR, generateInlineAsm` ...

Figure 2 shows the translation mechanism of `id = gpr(i) {J}`. We use function `evaluteId` to evaluate left value of *id*. As a result, it returns `VarRegId` containing register index, that stores the base address of identifier. For writing to *gpr* one should, obviously, evaluate right value of *id*.

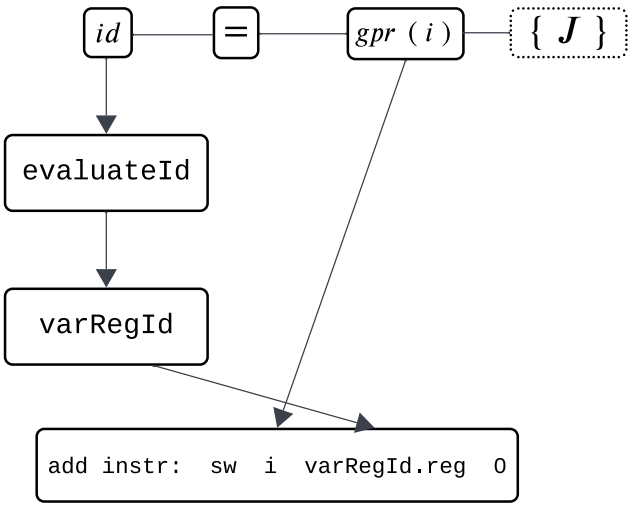


Figure 2: Read register content

As it is described on Code 8, first we block the register set that should not be accessed throughout the translation phase and after evaluating *id*, add *sw* instruction. Note that when writing to register – `addi i varRegId.register 0` is generated.

```
FUNCTION generateReadGPR(id, registerIdx):
  // 1. Initialize restricted register list with registerIdx
  restrictedRegisterList.add(idx)
  // 2. Check if J - is present.
  IF id.siblingsCount() > 5:
      // 3. Extract registers from J.
      FOR each j extracted in J:
        // 4. Update restrictedRegisterList with j
        restrictedRegisterList.add(j)
      END FOR
  END IF
  // 5. Block registers from restrictedRegisterList
  FOR reg in restrictedRegisterList:
    // 6. Block each restricted register
      occupy(reg)
  END FOR
  // 7. Finish translation with sw instruction
  addInstruction(Instruction.sw(idx, varRegId.register, 0))
  // 8. Free occupied registers for J
  FOR reg in restrictedRegisterList:
      // 9. Block each restricted register
      free(reg)
  END FOR
  // 10. Free register used in id translation
  free(varRegId.reg)
END FUNCTION
```

Code 8: generateReadGPR

Unlike reading or writing to registers, inline assembly itself provides MIPS instruction. Therefore, it does not require any further translation. We simply copy the the content under its derivation tree node - `<ASM>` as it is given at Code 9.

```
FUNCTION generateInlineASM(instrNode):
  // 1. create new instruction with content of instrNode
  instruction = instrNode.getBorderWord()
  // 2. add instruction to the instruction list.
  addInstruction.add(instruction)
END FUNCTION
```

Code 9: generateInlineASM

4.3. Void Function

4.3.1. Grammar

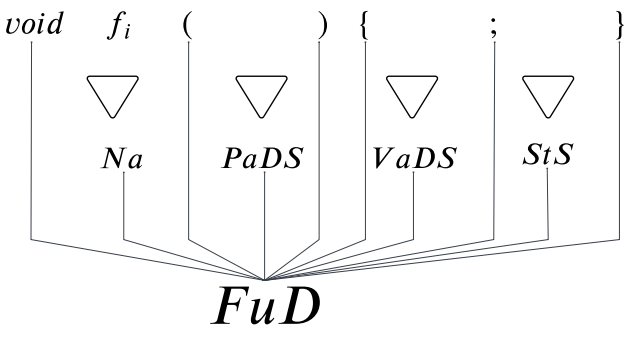


Figure 3: Void Function Derivation

Void functions, unlike ordinary ones, do not return any value and therefore possesses a predetermined *void* type. Consequently, their entire body can simply be described with <StS>. As regular functions, it may declare both – parameters to be passed upon function call and local variables in the beginning of function definition. Figure 3 depicts only one of the four possible void generation. Luckily, this change to the grammar did not become any source of an overhead to DK_1 test.

4.3.2. Derivation

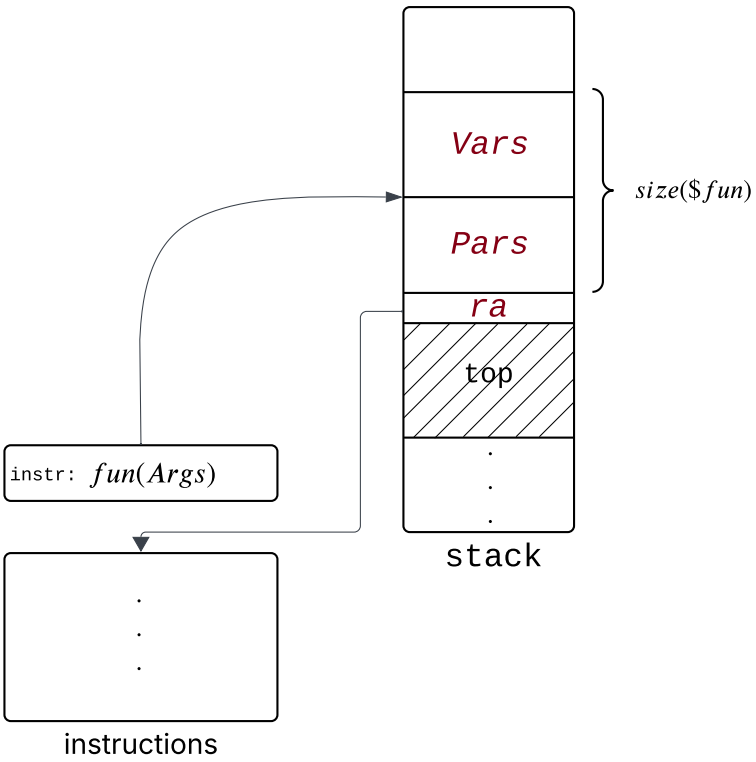


Figure 4: Void function derivation

Before actual jump into the function execution, first one should check available space on the stack, whether it can allocate the new function frame. In case there is not enough space, *syscall* interrupt is generated and to indicate its status $gpr(1)$ is updated with value 41. As before we keep `MemoryHelper.java` for that.
Figure 4 displays an extra word stored on stack indicating return address at which assembly program should continue execution. This is achieved with the aid of `jal` and the first instruction in function body translation. For more details, see Chapter 12.4 [1].

4.3.3. Implementation

When a void function declaration is detected at the `DTE <St>` border word, it requires to evaluate all arguments and initialize values of local variables (if they present) with zeros. This is usually done with helper functions `setParametersRegisterList, initializeLocalVariables`. Translation is completed by providing generation of function body – <StS> with `generateCodeVoidFunction` (see at Code 10).

```
FUNCTION generateCodeVoidFunction(call):
  // 1. generate instruction store return address on the stack
  addInstruction(Instruction.sw(31, 29, -(size(call) + 4))
  // 2. Extract body from function
  body = call.getFunction().getBody()
  // 3. generate translation for function body
  generateStS(body)
  // 4. Add instruction for returning from function
  addInstruction(Instruction.lw(1, 29, -(call.frameSize + 4)))
  // 5. Decrease stack pointer
  addInstruction(Instruction.addi(29, 29, -(call.frameSize + 4)))
  // 6. Generate instruction jumping to return address
  addInstruction(Instruction.jr(1))
END FUNCTION
```

Code 10: generateCodeVoidFunction

4.4. Examples

This chapter summarizes translation examples of *writems* and *copyms*. Throughout kernel programming, these helper functions are exploited to access disk.

- `void writems(uint x, uint a)` – stores value x in main memory at a
- `void copyms(uint a, uint b, uint L)` – copies L consecutive words from main memory starting at address a to main memory starting at address b

```
void writems(uint x, uint a){

    gpr(1) = x;
    gpr(2) = a {1};
    asm( sw 1 2 0 )
}
```

```
_writems:
sw 31 29 -12
addi 1 29 -8
lw 1 1 0
addi 1 1 0
addi 2 29 -4
lw 2 2 0
addi 2 2 0
sw 1 2 0
lw 1 29 -12
addi 29 29 -12
jr 1
```

Code 11: *writems* translation

```
void copyms(uint a, uint b, uint L){

        gpr(1) = a;
        gpr(2) = b {1};
        gpr(3) = L {1, 2};

        asm( blez 3 7 );
        asm( lw 4 1 0 );
        asm( sw 4 2 0 );
        asm( addi 1 1 4 );
        asm( addi 2 2 4 );
        asm( addi 3 3 -1 );
        asm( blez 0 -6 )
};
```

```
_copyms:
sw 31 29 -16
addi 1 29 -12
lw 1 1 0
addi 1 1 0
addi 2 29 -8
lw 2 2 0
addi 2 2 0
addi 3 29 -4
lw 3 3 0
addi 3 3 0
blez 3 7
lw 4 1 0
sw 4 2 0
addi 1 1 4
addi 2 2 4
addi 3 3 -1
blez 0 -6
lw 1 29 -16
addi 29 29 -16
jr 1
```

Code 12: *copyms* translation

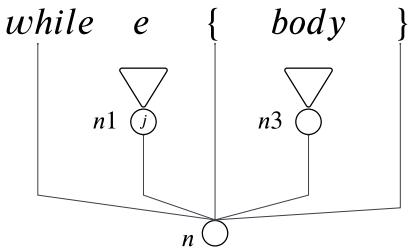
4.5. Corrections to Pre-Existing Bugs

While integrating our modifications, certain bugs were uncovered inside already existing code generation portions. Here, we underline those that we consider the most significant.

4.5.1. Macros Inside Conditional Statement

Translation process is usually simplified with *macros* — each grouping together sets of MIPS instructions that serve the same purpose. *macro* instructions are usually unfolded from MIPS to bytecode translation stage. For further details see Chapter 3.2.2,

For conditional statements: *while*, *if*, and *if else*, we evaluate the condition first and then generate a branch instruction with calculated offset (see Figure 5). Initially, these branches targeted in the wrong place because offset computation logic treated each macro as a single instruction. To handle this, *macro*'s full length must be included in the offset by maintaining a map - `Map<String, Integer> instructionsRealSize` - that stores actual number of MIPS instructions generated for each *macro*.




```
code (n1)
beqz j |code(n3) + 2|

code(n3)
blez 0 -(|code(n1) + |code(n3)| + 1)
```

Figure 5: *while* derivation

4.5.2. Constant Evaluation

Next, we handle the bug found in loading constant values. According to specifications given in Chapter 12.2.5 [1], constants generation of simple types such as: `bool, char, int, uint` should be achieved with *macro* instruction. In this case, *macro* loads upper and lower part of value separately with *lui* and *ori* (see Code 4). Initially, there was a temptation to use `addi i 0 val` to load constants, but since this is limited to only 16-bit immediate values, we replaced it with *macros* accordingly.

```
# Before (using addi, limited to 16-bit)
addi 1 0 1000

# After (using our macro for a 32-bit unsigned constant)
macro: gpr(1) = enc(1000, int)
```

Code 13: Constant evaluation

4.5.3. Array Element Generation

Last major bug was found in sub-variable generation from array. The derivation – `id -> id[e]` returns new ***Variable*** with following components:

- ***Base Address*** - evaluates base address of the sub-variable;
- ***Type*** - returns respective type that was used to define array-type;
- ***Displacement*** - computes displacement of `id[e]` inside `id` variable;

Some of these calculations were implemented incorrectly, causing sub-variable accessing to wrong memory locations. We reviewed all three components — base-address evaluation, element-type lookup, and displacement computation - so that the resulting ***Variable*** now accurately represents $id[e]$ and its memory layout.

5. Simulator

This chapter covers the design and implementation of a single-cycle MIPS simulator. The simulator runs a defined subset of the MIPS ISA (Table 1, Table 2, Table 3) and was built mainly as a debugging tool for verifying assembly programs during development. The behavior follows the ISA specification from Section 8.2 of "A Basic Sequential MIPS Machine" in [1].

MIPS simulators already exist. MARS [4] is probably the most widely used one. But using a general-purpose tool would have created friction: differences in how instructions are handled, how pseudo-instructions get expanded, and how labels are written would all have caused problems specific to our setup. Building our own meant we could match the exact ISA subset and assembly syntax we were working with. It also forced a clearer understanding of how hardware state and instruction execution actually relate to each other, which mattered for the rest of the project.

The rest of this chapter walks through the state model of the simulated machine, how the core simulation loop works, how key instruction groups were implemented, and how we checked the simulator's correctness using example programs. All the relevant classes are grouped under a package called *simulator*.

5.1. Architectural State Model

5.1.1. Configuration

The simulated machine's state is stored in a single structure called the `Configuration`. It holds everything an instruction can read from or write to at any point during execution. That means three things: the Program Counter, the General-Purpose Register file, and main Memory. These are shown in Figure 6.

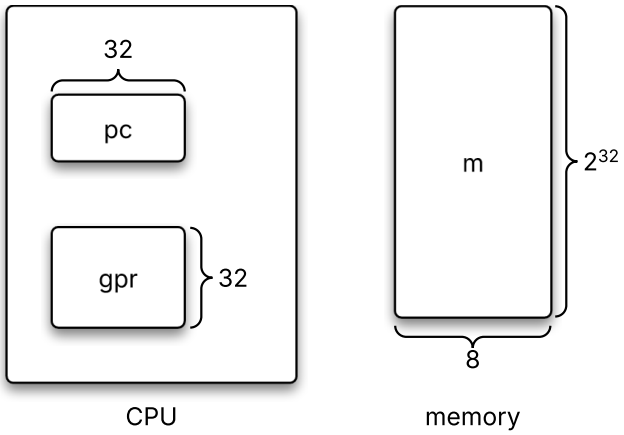


Figure 6: Visible data structures of MIPS ISA

In the Java implementation, these components live directly inside the `Configuration.java` class, which holds the full machine state throughout simulation.

```java
public class Configuration {
    private long pc;
    private final int[] gpr;
    private final Map<Integer, byte[]> memoryPaged;
    private boolean isRunning;

    public Configuration() {
        this.pc = 0L;
        this.gpr = new int[32];
        this.memory = new HashMap<>();
        this.isRunning = true;
    }

    // ... methods for getting/setting state ...
}
```

Code 14: `Configuration` java class

The class fields directly correspond to the MIPS architecture. The `pc` field is implemented as a 64-bit `long` to correctly represent the full range of a 32-bit unsigned address, as Java lacks a native unsigned 32-bit integer type. The `gpr` field is an array of 32 integers, directly modeling the MIPS general-purpose register file; the constant value of register `zero` is maintained by ensuring that any write operation to index 0 is ignored. The most significant design choice, however, lies in the implementation of the main memory.

5.1.2. Paged Memory

Simulating a full 32-bit address space directly would mean allocating an array of 2^{32} entries, which is not practical. Instead, we used a sparse memory model built on a `java.util.HashMap<Integer, byte[]>` that only allocates memory when something is actually written to it.

The address space is split into fixed 4KB blocks called pages. The map acts as a page table, where each entry maps a page ID to a `byte[]` of 4096 bytes. A page only gets created the first time a program writes to an address inside it.

Any 32-bit address breaks into two parts: the upper 20 bits identify the page, and the lower 12 bits give the offset within that page's array. Reads and writes inside an already-allocated page are then just direct array accesses. If a read hits a page that was never written to, the simulator returns zero, which matches the behavior of zeroed physical memory without needing to pre-allocate anything.

5.1.3. Memory Access and Byte Ordering

MIPS is byte-addressable, but instructions like `lw` and `sw` work on 32-bit words. So the simulator needs to pack four bytes into a single word on reads, and unpack a word back into four bytes on writes. This is done in the `getWord` and `setWord` methods inside Configuration.

```java
public int getWord(int address) {
    return  ((getByte(address) & 0xFF) << 24) |
            ((getByte(address + 1) & 0xFF) << 16) |
            ((getByte(address + 2) & 0xFF) << 8) |
            ( getByte(address + 3) & 0xFF);
}

public void setWord(int address, int value) {
    setByte(address,     (byte) ((value >>> 24) & 0xFF));
    setByte(address + 1, (byte) ((value >>> 16) & 0xFF));
    setByte(address + 2, (byte) ((value >>> 8) & 0xFF));
    setByte(address + 3, (byte) (value & 0xFF));
}
```

Code 15: `getWord`, `setWord` methods

The tricky part is Java's handling of signed bytes. In `getWord`, each byte gets masked with `& 0xFF` before being combined, otherwise Java sign-extends it when promoting it to an integer and the result comes out wrong. In `setWord`, the unsigned right-shift `>>>` is used instead of `>>` to pull out the right eight bits without dragging the sign bit along.

5.2. Simulation Cycle

The simulator runs one instruction at a time through a loop that mimics the classic Fetch-Decode-Execute cycle. It keeps going until either the program runs out of instructions or a `syscall` tells it to stop.

Formally, the MIPS ISA can be described as a state transition function δ that takes one machine configuration and produces the next, as defined in chapter 8.2.1 of [1]. Each instruction execution is a single step of that transition:

$$c' = \delta(c, 0)$$
$$c^{i+1} = \delta(c_i, 0),$$

In the Java implementation, this is what `simulateStep` does. It takes the current `Configuration`, decodes and executes one instruction, and updates the state in place, directly implementing δ.

The simulation can be started in two ways: either by passing in raw bytecode, which gets loaded into a fresh configuration, or by handing it a configuration that already has a program loaded. The first approach looks like this:

```
FUNCTION simulate(program_bytecode):
  config = CREATE_NEW_CONFIGURATION()
  FOR i FROM 0 TO program_bytecode.length - 1:
    address = i * 4
    config.setWord(address, program_bytecode[i])
  END FOR

  WHILE config.isRunning() AND config.hasWordAt(config.getPC()):
    instruction_word = config.getWord(config.getPC())
    simulateStep(config, instruction_word)
  END WHILE

  RETURN config
END FUNCTION
```

The **Fetch** phase reads the 32-bit instruction word at the address the Program Counter currently points to, matching the formal definition $I(c) = c.m_4(c.\text{pc})$.

In the **Decode** phase, `simulateStep` extracts the 6-bit opcode from bits 31-26 of that word, as defined by $\text{opc}(c) = I(c)[31:26]$:

```
int opcode = (instructionWord >>> 26);
```

The opcode determines the instruction format. `simulateStep` acts as a dispatcher, routing to the right handler based on the predicates from [1]:

$$\text{rtype}(c) \equiv \text{opc} = 0^*0^4$$

$$\text{jtype}(c) \equiv \text{opc} = 0^41^*$$

$$\text{itype}(c) \equiv \overline{\text{rtype}(c) \vee \text{jtype}(c)}$$

```
   void simulateStep(Configuration configuration, int instructionWord) {
     int opcode = (instructionWord >>> 26);

     if (opcode == 0b000000 || opcode == 0b010000) {
         simulateRType(configuration, opcode, instructionWord);
     } else if (opcode == 0b000010 || opcode == 0b000011) {
         simulateJType(configuration, opcode, instructionWord);
     } else {
         simulateIType(configuration, opcode, instructionWord);
     }
 }
```

Each handler then parses the remaining fields, things like `rs`, `rt`, `rd`, or an immediate value, according to the layouts in Figure 1. The general structure looks like this:

```
FUNCTION simulateSpecificType(config, opcode, instructionWord):
  operands = PARSE_OPERANDS(instructionWord)
  mnemonic = MnemonicUtils.getMnemonic(opcode, fun, ...)
  instruction = InstructionRegistry.getExecutableInstruction(mnemonic, operands)
  instruction.execute(config)
END FUNCTION
```

5.3. Instruction Registry

The `InstructionRegistry` sits at the center of both the assembler and the simulator. It uses a map-based factory pattern, but keeps two separate maps: one that creates instruction objects for encoding (`encoderInstructionMap`), and one that creates objects for execution (`executableInstructionMap`).

Both maps are populated once at startup. For each mnemonic, say “add”, two factory lambdas get registered. The first creates a basic instruction object the assembler uses to produce bytecode. The second creates an object that carries the actual simulation logic inside its `execute` method. When the simulator calls it, that method updates the `Configuration` state according to the ISA spec.

Keeping the two maps separate means the dispatch logic does not need to know anything about how individual instructions work. Adding a new instruction is just a matter of registering it in both maps. See Code 30.

5.4. C0-Programs Simulation

Although we are unable to provide encoded abstract C0 configuration, correctness of large translation portions can still be checked with the simple simulator. Currently, we can fully simulate *virtual user*'s programs that do not access *disk* and *special purpose registers*, nor lead to *interrupts*. Below, we briefly describe our methodology for validating translation.

After translating high-level program, we initialize memory component of configuration - $c.m$ with machine code. Note that, since both - `j` and `jal` instruction performs *pc* relative jumps, one may put *Bytecode* anywhere in region - $[a : b]$. Additionally, we also specify memory portion - *stack* - for allocating global variables and function frames. For further details see – `Simulator.java`

```
int a = 0; // Beginning of translated code
int b = 4 * K; // Code should fit into a single page (1024 words)

int SBASE = 5 * K; // Beginning of stack
int SMAX = 9 * K; // Size of the stack is also fixed with 1024 words

int HBASE = 10 * K; // Beginning of heap
int HMAX = 14 * K; // End of heap, total 1024 words as well.
```

Code 16: Simulator's configuration

Correctness of the simulation relies on 2 key factors:

1. values of variables defined in program, and
2. status of $gpr(1)$ after completion of simulation.

We finish each testing program's *main* function with *return* 1. Consequently, its translation ends up with:

```
macro: gpr(1) = enc(1, int)
syscall
```

Code 17: Translation of *main*'s return

In our simulator - `syscall` interrupt is used to complete execution. Once halted, we inspect $gpr(1)$ and selected global/local variables to deduce whether simulation was held correctly or not. By doing so, we are also able to validate independent translation regions.

5.5. Translation-to-Simulation: Challenges & Solutions

After enabling the simulation of actual C0 programs, we encountered several minor challenges. Although, the code generation strictly follows book specifications, simulation revealed existence of necessary adaptions to the compiler, that otherwise will fail simulation of programs.

5.5.1. Initialize Stack Pointers

To track stack pointers throughout code generation, it requires to precede each translated program with instructions that explicitly initializes stack, heap and base pointer registers respectively. Additionally, it also zero-initialize all global memory variables - ***gm***. For achieving the second goal, *macro* zero is exploited.

```
# gpr(28) - specifies base address
# gpr(1) - number of words that assign 0.
zero(28, 1)
```

Code 18: Initialize *gm* with Zeros

After executing this *macro*, register $gpr(28)$ no longer contains the original *sbase* value, since the *zero*'s expansion overwrote it. Consequently, any subsequent access to global variables will use wrong base address. To prevent this, we insert an extra instruction immediately after zero-initializing ***gm*** that restores $gpr(28)$ to its correct *sbase* value.

```
macro: gpr(30) = enc(hbase, uint)
macro: gpr(28) = enc(sbase, uint)
addiu 29 28 size($gm) + size($main) + 8

subu 1 29 28
srl 1 1 2
macro: zero(28, 1)

macro: gpr(28) = enc(sbase, uint)
```

Code 19: Initialize *gm* and pointer registers

5.5.2. Function call

Originally, *function-call* generation first increments the stack pointer by the size of the function frame, and only then computes the base address of the return-destination variable and evaluates the arguments for the *function*. However, adjusting the stack pointer in the first place, disrupts later calculations — both arguments and the return destination rely on the original stack-pointer value.

There are two intuitive solutions for that, we describe each of them, and explain the one we adopted to continue work with.

1. Re-order steps: evaluate arguments first and hold their values into registers, adjust stack-pointer and only after store previously evaluated arguments to the newly allocated function frame. In this scenario, maximum number of arguments is limited 32 available registers.
2. Keep the original sequence, but instead modify the generation of *arguments* by explicitly adjusting it to *old frame*. In this case one should update – `ExpressionEvaluation.java`.

We settled on solution (1), as it merely changes the order of the steps without altering any code modules. On Code 20 we describe generation according to Figure 7.

```
// Return destination variables' base address computation
code (n_0)

// Evaluate arguments for all 1 <= y <= p
code (n_y)

// Adjust stack pointer

// Store base address on the stack frame
sw j spt -($size(g) + 4)

// Store each argument at its proper offset on the stack frame
sw j_y spt -$size(g) + displ(e_y, $g)
```

Code 20: Function call

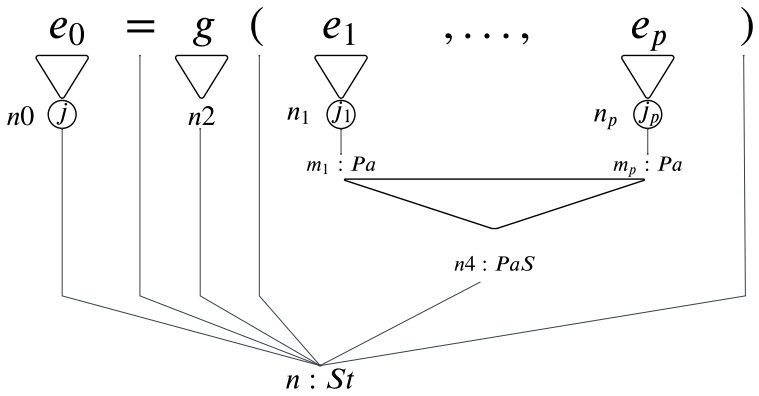


Figure 7: Function call

5.6. Verification

To test both the C0 compiler and the MIPS simulator, we wrote over 70 C0 test programs covering a broad range of language features: integer and unsigned integer arithmetic, pointers, structs, loops, conditionals, and recursive calls.

Each program is self-verifying. It runs some operations, puts the result somewhere predictable in memory or a register, then exits with a status code of 1 through a `syscall`. If the simulator ends up with the wrong values, the test fails.

The whole process is automated with `JUnit`. For each program, the test does the following:

1. Compiles the C0 source to MIPS bytecode using the compiler from [2].
2. Creates a fresh `Configuration`.
3. Loads the bytecode into memory.
4. Runs `simulate()`.
5. Checks that the registers and memory locations match the expected values after the simulation stops.

This covers the full pipeline from source code to execution in a single test run. All test programs live under the `test/c0 programs` package.

6. Basic OS-Support

6.1. Overview

To support the execution of multiple *users'* processes, we introduce a minimal operating-system kernel, whose services should later also be deployed on FPGA board. This chapter presents kernel's implementation details, which comprises two primary components:

1. ***bootloader*** - detects *reset* signal and fetches translated kernel code from disk.
2. ***runvm*** - kernel primitive for running *virtual user's* code on *host* with transparent handling of invalid page-fault interrupt.

For delivering *bootloader*, we reserve the starting portion of main memory – *ROM* and initialize it alongside with *bootloader's* machine code. On power-up - or upon any interrupt signal – control immediately switches to the *ROM* region. We assume that *reset* interrupt is raised only during the first hardware cycle. Once it is obtained, *bootloader* fetches translated kernel code from disk into a fixed region of main memory and begins to execute it. If *reset* is not detected – indicating that kernel is already present – *bootloader* finishes execution with direct jump inside *runvm* primitive. The latter saves current *user's* process and handles transparently any subsequent invalid-page-fault interrupt.

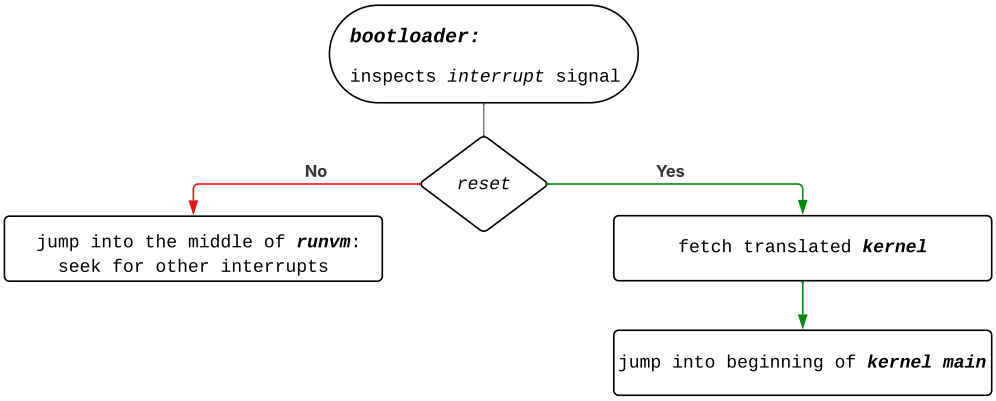


Figure 8: Interrupt detection

Switching between *user* and kernel modes requires saving and restoring each *user's* configuration portions. Furthermore, *users'* memory accesses must be translated through MMU. To support above, kernel usually provides globally defined data structures for its *users*. After fetching, kernel initializes each data block with corresponding values and then enters its main scheduling loop:

1. select next *user* process to run - via *scheduler*
2. restore respective *user's* configuration and execute its code - via *runvm*.

Only invalid-page-fault interrupt is handled within *runvm*. Normally, *dispatcher* provides treatment for other interrupts. However, to keep kernel simple and remain focused on our goal - running multiple *users'* processes on FPGA board, we consequently leave out *dispatcher* from main loop (Code 21).

```
1 [...] // initialize data structures
2 while true
3 {
4   scheduler();
5   runvm()
6 }
```

Code 21: Kernel

The implementation details rely directly on the specifications given in Chapter 15 [1]. In the following subsections, we will cover only those parts that are essential for kernel programming. Detailed description of disk-access management is therefore omitted. To see a full treatment of it, refer to Chapter 14.3 [1].
We begin this section by presenting kernel's core components:

- memory layout and initialization of data structures
- simple *scheduler* to support simulating more than a single *user's* process, and
- description of *runvm* primitive tasks.

In the latter half, we go back to the *bootloader*, detailing its design and the major challenges encountered during its development.

6.2. Basic Kernel: Technical Aspects

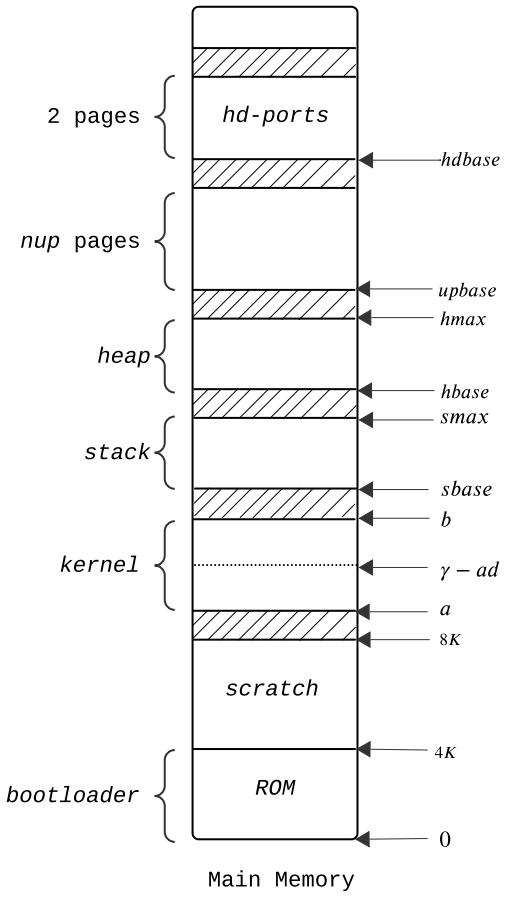


Figure 9: *main memory* layout

Before diving into the design details below, we will first review the key technical decisions made for kernel programming. The entire kernel is written in "C0 + Assembly" within a single program - `AbstractKernel.c0`. This approach greatly simplifies build process - there is no need to generate code portions separately - but it comes at the cost of generality. Because C0 does not support parameter-sized data structures, we are forced to use constant-sized structures and maintain extra global variables to track their lengths. At present, kernel is expected to simulate two different *user's* tasks, anyone wishing to increase this number should explicitly modify - `AbstractKernel.c0`.
Similar approach is exploited for *bootloader* as well, both - `AbstractKernel.c0`, `FetchKernel.c0` appear in Appendix - Section A.5, Section A.6, alongside with the MIPS instructions used to initialize *ROM* - `RPM.asm` - Section A.7.

6.3. *Main Memory System* Layout

To ensure that the *bootloader*, kernel, and *disk-access* primitives share consistent memory map, we first define *host machine's* memory system layout (see Figure 9). Below, K defines the number of *words* into a single page - 1024.

The reserved regions are:

- ***ROM*** - $[0 : 4K]$ - contains *bootloader*. At start-up, only *ROM* is present.
- ***scratch*** - $[4K : 8K]$ - used for saving and restoring register contents. (Chapter 15.5.1 [1])
- ***kernel*** - $[12K : 20K]$ - holds two pages of translated kernel code - (1,629 MIPS instructions).
- ***stack*** - $[24K : 32K]$ - allocates data structures and function frames for kernel.
- ***heap*** - $[36K : 40K]$ - reserved region for heap-allocated variables inside kernel (current implementation does not utilize any dynamic heap variables).
- ***upbase*** - $[44K : 52K]$ - provides memory space, where *users'* virtual pages are allocated on *host* machine. Presently, it can hold only — 2 pages. This region is accessed during *ipf*-page fault interrupt handling.
- ***hd-ports*** - $[56K : 64K]$ - two pages mapped to disk-ports (Chapter 14.3 [1]).

Note that address ranges are given in bytes, and chosen so that each region is page aligned.

6.4. Kernel Data Structures & Initialization

Having defined memory layout, we now begin to describe *kernel's* core data structures - global variables and arrays allocated on *stack*. These structures are usually utilized to provide two major goals:

1. performing address translation and tracking virtual pages for each *user*
2. saving and restoring *user* processes between switching modes.

Below, we explain how these structures are used and show their initialization routine. For more details, see Appendix - Section A.5.

6.4.1. Address Translation Support

To enable *virtual users* perform their corresponding memory accesses, address translation is required. Normally, each *user* maps its virtual pages into *hosts*' physical memory, providing page tables. When page is not yet loaded in the table, *ipf*-page fault is raised (see Chapter 14.2 [1]). This interrupt is handled transparently by *runvm* primitive.

For managing page tables, kernel consequently introduces two global arrays:

- *PTA* - (flat page table array) comprising map of all *users'* pages tables.
- *PTOI* - (page table origin index) indicating for each *user* the beginning of their page table map in *PTA*.

```
1  typedef uint[PTASIZE] PTAt; // mapping of all users' virtual pages
2  typedef uint[p+1] PTOIt; // size of p + 1 - for better indexing
3  ...
4  PTAt PTA;
5  PTOIt PTOI;
```

At present, maximum number of pages each *user* can allocate is specified by global variable *PTL*, therefore -

$$PTASIZE = p * PTL.$$

Since no virtual pages are loaded at startup, *PTA* - array should be initialized with 0's. We don't provide any explicit C0 code portion for that, as it will implicitly be zero-initialized in the beginning of kernel's execution.

As for *PTOI* – first *user* should start its page table in the beginning of *PTA*, and any subsequent *user* should continue after contiguous block of *PTL* entries in the table.

```
i = 1u;
while i<p+1u {
     PTOI[i] = (i-1u)*PTL;
     i = i + 1u
};
```

6.4.2. Process Control Block

For storing *users'* CPU configuration - both *gpr*, *spr* - on *host* machine, we define global array of process control blocks - *PCB*.

```
typedef uint[32] u; // gpr
typedef uint[8] v; // spr
typedef struct {u GPR; v SPR} pcb; // gpr + spr
typedef pcb[p+1] PCB_t; // for p - user and kernel
...
PCB_t PCB;
```

When restoring current *user* process, kernel loads its saved registers from `PCB[CP]`. To guarantee correct control switching, we must initialize each - `PCB[i]` accordingly. Below is the initialization loop for each *users'* four key *spr*:

```
i = 1u;
while i<p+1u {
      PCB[i].SPR[1] = 4294967295u;
      PCB[i].SPR[3] = 0u;
      gpr(1) = PTA[PTOI[i]]&;
      PCB[i].SPR[5] = gpr(1);
      PCB[i].SPR[6] = PTL;
      i = i + 1u
};
...
```

- $spr(1)$ — exception status register — restores status of maskable interrupts. Initialized to all 1's, as a result, maskable interrupts are not disabled by default.
- $spr(3)$ — exception *pc* — specifies program counter, where *user* should begin its execution. Set to 0 so that *user's* program simulation starts at the beginning of its memory.
- $spr(5)$ — page table origin — indicates the beginning of *user's* page table array portion inside *PTA*. Since *PTOI* manages this, we would simply load the base address of - `PTA[PTOI[i]]&`. Note that to achieve this, mixed C0 syntax is needed, as type of `PCB[i].SPR[5]` is - *uint*.
- $spr(6)$ — page table length — defines maximum number of *virtual pages* that *user* can allocate on *host*. Global variable *PTL* fixes this value and $spr(6)$ does not change after its initialization.

Unlike data structures supporting address translation for *users*, an extra *PCB* block is allocated for *host*. However, it is usually affected while switching between processes by *macros* – *restore-user* and *save-user*, and therefore, does not require any further initialization.

6.5. Simple Scheduling Mechanism

For scheduling, simple round-robin approach is used. During each iteration of infinite loop, scheduler selects current *user* process. As long as *user* continues running - or raises only invalid-page-fault interrupt - control goes back to it. Once they suspend, *scheduler* advances to the next *user*.

```
void scheduler(){

    if (CP==p) {CP = 1u} else {CP = CP+1u} // arbitration
};
```

6.6. Kernel Primitive – *runvm*

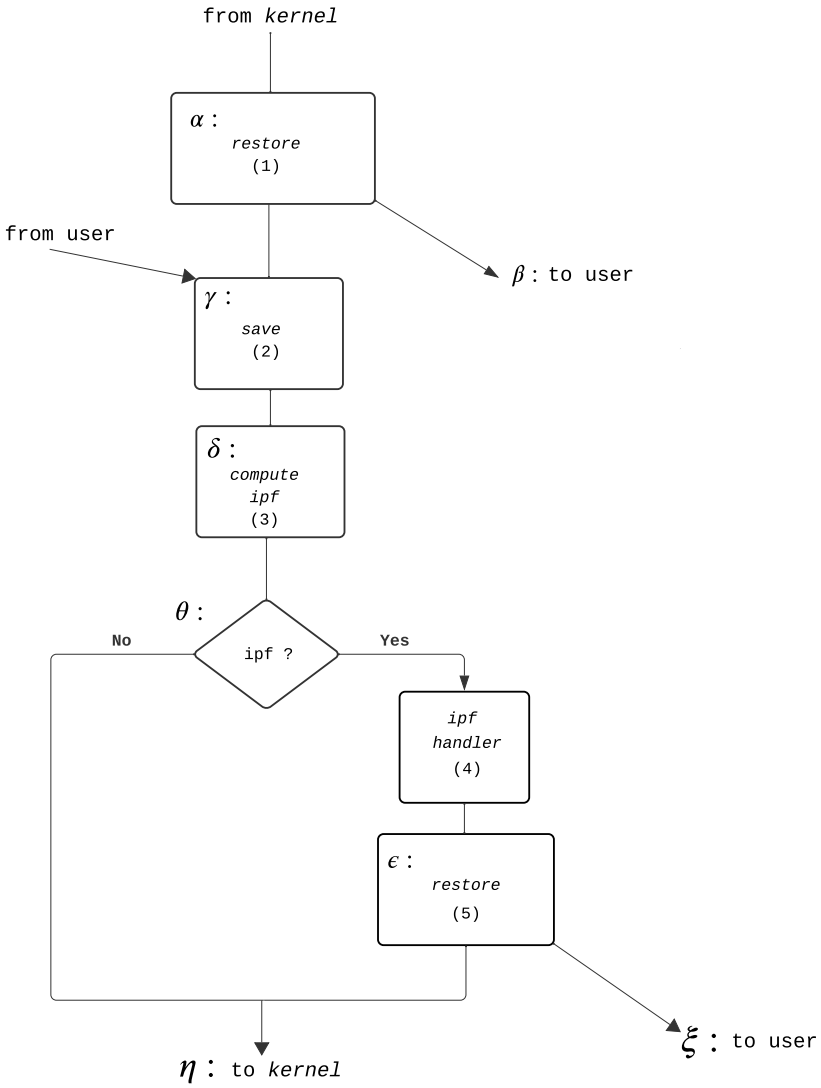


Figure 10: *runvm* Primitive

After scheduling current *user* process - *CP*, control switches from kernel to *user* via *runvm* primitive, which additionally detects *ipf*-page fault and invokes *ipfHandler* function to resolve it. This interrupt handling mechanism follows the direct specifications given in Chapter 15.6 [1], without any further modifications. Thus, we would not include it here, its construction details can be seen in Appendix - Section A.5.
As it is depicted on Figure 10 *runvm* function's control flow consists of the following steps:

1. Switch from kernel to *user* in the beginning of *runvm – restore-user*.
2. Saving a *user* task and returning control to the kernel after detecting an interrupt - handled by - *save-user*

3. Testing after an interrupt whether an *ipf*-page fault has to be handled by *runvm*. Here, we generate explicit *ipf* detection logic directly within *runvm*.
4. Transparent handling of *ipf*-page fault by function call *ipfHandler*.
5. Process switch from kernel to *user* after handling of an *ipf*-page-fault, achieved similarly with macro - *restore*.
6. Exit to the kernel after the suspension of *user* code.

Note that, when switching modes, we need to ensure that the *virtual user's* configuration - *gpr*, *spr* - can correctly be loaded to their physical counterparts. Consequently, in the beginning of restoring *user* process, we first store the base address of its process-control block - `PCB[CP]` into scratch memory. We accomplish this with the instruction:

```
// Note that PCB[CP].GPR[0] and PCB[CP] share the same base address
gpr(1) = PCB[CP].GPR[0]&;
asm( sw 1 0 4K+4 ) // first instruction of macro - restore-user
...
```

Since the translation of first statement depends on the kernel's current configuration, we leave out it from the *restore-user* macro and provide explicitly inside the *runvm* primitive. For full details, see Code 24.

```
void runvm(){
        // initialize register 1 to restore correct user - CP
        gpr(1) = PCB[CP].GPR[0]&;
        asm( macro: restore-user ); // macro - restore
        asm( macro: save-user ); // macro - save

        // ipf-page fault detection
        IL = 0u;
        i = 1u;
        found = 0u;
        while found==0u {
                IL = IL+1u;
                i = 2u*i;
                gpr(1) = i;
                gpr(2) = PCB[CP].SPR[2] {1};
                asm( and 3 1 2 );
                found = gpr(3)
        };

        EVA = 0u;
        if IL==17u {EVA = PCB[CP].SPR[3]};
        if IL==20u {EVA = PCB[CP].SPR[4]};

        gpr(1) = EVA;
        asm( srl 1 1 12 );
        EVPX = gpr(1);
        ptle = (EVPX>=PCB[CP].SPR[6]);
        ipf = (IL==17u||IL==20u)&&!(bool)ptle;
        // in case ipf-page fault, handle it, restore user.
        if (bool)ipf {
            ipfHandler();
            gpr(1) = PCB[CP].GPR[0]&;
            asm( macro: restore-user )
          }
};
...
```

Code 24: Primitive - *runvm*

6.7. Bootloader

We conclude this chapter by outlining *bootloader's* primary objective and the challenges we faced. Upon any interrupt, control is immediately given to the ROM region, where *bootloader* inspects the interrupt type. If *reset* is detected, it loads the translated kernel code from disk and then transfers control to the kernel to begin execution. If no *reset* is present - indicating the kernel is already in place - *bootloader* jumps directly to the middle of the *runvm* primitive, which saves *user* configuration and continues to monitor for invalid-page-fault interrupts. For more details, see Chapter 15.5.6 [1]

```
macro: ssave(1)
movs2g 2 1
andi 1 1 1
blez 1 len(KernelFetch) + 2

# Translation of KernelFetch C0 program
j _beginning of kernel

macro: srestore(1)
j _user save portion in runvm
```

6.7.1. Interrupt Signal Inspection

In order to detect whether *reset* is raised or not, one should normally use *host's* general purpose registers. We load exception-cause *spr* into - $gpr(1)$. However, since register 1 may still hold a value from the previously running *user* – and that value is not yet saved in `PCB[CP]` – we must save it first on *scratch* memory. On startup, when no *user* process exists, $gpr(1)$ contains no meaningful data, so we only restore it, if *reset* not is detected.

6.7.2. Fetching Kernel Code

After observing *reset*, it start execution of program - `FetchKernel.c0`. It is written in "C0 + Assembly" to load kernel code from disk into predetermined region on *host's* memory layout (Figure 9). Although `FetchKernel.c0` uses *host stack* for its own function call frames and variables, it runs only once at startup when there is no kernel loaded and therefore, does not alter the kernel's configuration and its data structures. The entire description of this program is in Appendix - Section A.6.

```
SPX = 0u;
PPX = a/4096u;
L = (b-a+4u)/4096u;

while L>0u {
      readdisk(PPX, SPX);
      SPX = SPX + 1u;
      PPX = PPX + 1u;
      L = L-1u
};
```

Code 25: Loading kernel code

At boot time, kernel is expected to be stored in the beginning of *swap memory* (disk) and is fetched one page at a time into the region - $[\boldsymbol{a} : \boldsymbol{b}]$ (see Code 25). Local variables – `SPX,PPX` are initialized with the starting page of disk and $[\boldsymbol{a} : \boldsymbol{b}]$ region, respectively. In this case, it is essential that both $\boldsymbol{a}, \boldsymbol{b}$ are page aligned (Figure 9).

Note that standard C0 programs usually end with *return* statement, we get it rid of it here, when translating from C0 to MIPS code, and instead of generating *syscall* instruction accordingly, execution continues with jump to the beginning of kernel. For full MIPS code that initializes *ROM* see Appendix - Section A.7

6.7.3. Adjusting Jump Lengths

As specified in Chapter 2 our MIPS provides *pc* relative jumps. Therefore, to generate - `j` instruction into predetermined address, we should take into consideration the current value of *pc*.

Consequently, at the end of kernel-loading routine - rather than jumping into $\boldsymbol{a}$, we explicitly evaluate current *pc* and specify offset as: $\boldsymbol{a - pc}$.

The similar approach of offset computation is used for jumping inside the *runvm* primitive, with one additional step - we are manually looking for the statement $\boldsymbol{\gamma}$ **- *macro: save-user*** (see Figure 10) in *runvm* and use it as a target address.

7. Conclusion: Limitations & Future Work

7.1. Overview

This thesis covers the design and implementation of a toolchain that takes C0 source code and produces executable MIPS bytecode, ending with a basic OS kernel that runs the result on hardware. The pipeline goes all the way from a high-level language down to a managed runtime.

The main things we built:

- An extension of the C0 compiler that adds a "C0 + Assembly" dialect, making kernel-level programming possible.
- A standalone MIPS assembler that handles forward-label references and macro expansion.
- A MIPS simulator with a demand-paged memory model, which turned out to be essential for debugging the translation pipeline.
- A basic OS kernel with a bootloader and a `runvm` primitive that can run virtual user processes and handle page-fault interrupts.

The full system was tested across a range of C0 programs and behaved correctly throughout.

7.2. Limitations

A few limitations are worth noting, most of them caused by hardware and software development happening at the same time.

- **Hardware unavailability:** The OS kernel was never tested on the actual FPGA board. The hardware designs for disk access, interrupt handling, and the MMU were not finished in time, so those parts of the kernel could only be verified in simulation.
- **Simplified kernel services:** The kernel is intentionally minimal. Scheduling is round-robin, and only invalid-page-fault interrupts are handled.
- **Fixed-size kernel structures:** Several kernel data structures have fixed sizes due to constraints in C0. Adding support for more user processes would mean editing the kernel source by hand.

7.3. Future Work

The most pressing next step is running the kernel on real hardware. That would put the disk I/O routines, the bootloader, and the interrupt handling under conditions the simulator cannot fully replicate.

Beyond that, the C0 compiler could be extended to cover more language features, and the assembler could be updated to recognize a wider set of standard MIPS pseudo-instructions, which would make it easier to use with existing MIPS code.

Bibliography

[1] Wolfgang J. Paul, Christoph Baumann Petro, and Lutsyk Sabine Schmaltz, *System Architecture: An Ordinary Engineering Discipline*. 2016.

[2] Toma Pirtskhelani and Irakli Khotchava, "Compiler Development for LR(1) Grammars: DK1 Test, Parsing, and Code Generation with Application to the C0 Language."

[3] Michael Sipser, *Introduction to the Theory of Computation, 3rd ed.*

[4] Kenneth Vollmar and Pete Sanderson, "MARS: MIPS Assembler and Runtime Simulator."

Appendix

A.1 C0 Grammar

$\langle Di \rangle$	$\longrightarrow 0 \mid \ldots \mid 9$	digit
$\langle DiS \rangle$	$\longrightarrow \langle Di \rangle \mid \langle Di \rangle \langle DiS \rangle$	digit sequence
$\langle Le \rangle$	$\longrightarrow a \mid \ldots \mid z \mid A \mid \ldots \mid Z \mid _$	letter
$\langle DiLe \rangle$	$\longrightarrow \langle Le \rangle \mid \langle Di \rangle$	alphanumeric symbol
$\langle DiLeS \rangle$	$\longrightarrow \langle DiLe \rangle \mid \langle DiLe \rangle \langle DiLeS \rangle$	sequence of symbols
$\langle Na \rangle$	$\longrightarrow \langle Le \rangle \mid \langle Le \rangle \langle DiLeS \rangle$	name
$\langle C \rangle$	$\longrightarrow \langle DiS \rangle \mid \langle DiS \rangle u \mid$ null	int/uint/null pointer constant
$\langle CC \rangle$	$\longrightarrow$ '␣' $\mid \ldots \mid$ '~'	*char*-constant with ASCII code
$\langle BC \rangle$	$\longrightarrow$ true $\mid$ false	*bool*-constant
$\langle id \rangle$	$\longrightarrow \langle Na \rangle \mid \langle id \rangle . \langle Na \rangle \mid \langle id \rangle [\langle E \rangle] \mid$ $\langle id \rangle * \mid \langle id \rangle \&$	identifier
$\langle F \rangle$	$\longrightarrow \langle id \rangle \mid -_1 \langle F \rangle \mid (\langle E \rangle) \mid \langle C \rangle$	factor
$\langle T \rangle$	$\longrightarrow \langle F \rangle \mid \langle T \rangle * \langle F \rangle \mid \langle T \rangle / \langle F \rangle$	term
$\langle E \rangle$	$\longrightarrow \langle T \rangle \mid \langle E \rangle + \langle T \rangle \mid \langle E \rangle - \langle T \rangle$	expression
$\langle Atom \rangle$	$\longrightarrow \langle E \rangle > \langle E \rangle \mid \langle E \rangle >= \langle E \rangle \mid$ $\langle E \rangle < \langle E \rangle \mid \langle E \rangle <= \langle E \rangle \mid$ $\langle E \rangle == \langle E \rangle \mid \langle E \rangle != \langle E \rangle \mid \langle BC \rangle$	atom
$\langle BF \rangle$	$\longrightarrow \langle id \rangle \mid \langle Atom \rangle \mid !\langle BF \rangle \mid (\langle BE \rangle)$	Boolean factor
$\langle BT \rangle$	$\longrightarrow \langle BF \rangle \mid \langle BT \rangle$ && $\langle BF \rangle$	Boolean term
$\langle BE \rangle$	$\longrightarrow \langle BT \rangle \mid \langle BE \rangle \mid\mid \langle BT \rangle$	Boolean expression
$\langle St \rangle$	$\longrightarrow \langle id \rangle = \langle E \rangle \mid \langle id \rangle = \langle BE \rangle \mid \langle id \rangle = \langle CC \rangle \mid$	assignment statement
	if $\langle BE \rangle$ $\{\langle StS \rangle\} \mid$	conditional statement (if-then)
	if $\langle BE \rangle$ $\{\langle StS \rangle\}$ else $\{\langle StS \rangle\} \mid$	conditional statement (if-then-else)
	while $\langle BE \rangle$ $\{\langle StS \rangle\} \mid$	loop
	$\langle id \rangle = \langle Na \rangle (\langle PaS \rangle) \mid \langle id \rangle = \langle Na \rangle () \mid$	function call
	$\langle id \rangle =$ new $\langle Na \rangle *$	allocation of memory
$\langle rSt \rangle$	$\longrightarrow$ return $\langle E \rangle \mid$ return $\langle BE \rangle \mid$ return $\langle CC \rangle$	return statement
$\langle Pa \rangle$	$\longrightarrow \langle E \rangle \mid \langle BE \rangle \mid \langle CC \rangle$	parameter
$\langle PaS \rangle$	$\longrightarrow \langle Pa \rangle \mid \langle Pa \rangle, \langle PaS \rangle$	parameter sequence
$\langle StS \rangle$	$\longrightarrow \langle St \rangle \mid \langle St \rangle; \langle StS \rangle$	statement sequence
$\langle prog \rangle$	$\longrightarrow \langle TyDS \rangle; \langle VaDS \rangle; \langle FuDS \rangle \mid$	*C*0-program
	$\langle VaDS \rangle; \langle FuDS \rangle \mid$	no type declaration
	$\langle TyDS \rangle; \langle FuDS \rangle \mid$	no variable declaration
	$\langle FuDS \rangle$	only function declarations
$\langle TyDS \rangle$	$\longrightarrow \langle TyD \rangle \mid \langle TyD \rangle; \langle TyDS \rangle$	type declaration sequence
$\langle TyD \rangle$	$\longrightarrow$ typedef $\langle TE \rangle$ $\langle Na \rangle$	type declaration
$\langle Ty \rangle$	$\longrightarrow$ int $\mid$ bool $\mid$ char $\mid$ uint $\mid \langle Na \rangle$	type names
$\langle TE \rangle$	$\longrightarrow \langle Ty \rangle [\langle DiS \rangle] \mid$	type expression, array
	$\langle Ty \rangle * \mid$ struct $\{\langle VaDS \rangle\}$	pointer, struct
$\langle VaDS \rangle$	$\longrightarrow \langle VaD \rangle \mid \langle VaD \rangle; \langle VaDS \rangle$	variable declaration sequence
$\langle VaD \rangle$	$\longrightarrow \langle Ty \rangle$ $\langle Na \rangle$	variable declaration
$\langle FuDS \rangle$	$\longrightarrow \langle FuD \rangle \mid \langle FuD \rangle; \langle FuDS \rangle$	function declaration sequence
$\langle FuD \rangle$	$\longrightarrow \langle Ty \rangle$ $\langle Na \rangle$ $(\langle PaDS \rangle)\{\langle VaDS \rangle; \langle body \rangle\} \mid$	function declaration
	$\langle Ty \rangle$ $\langle Na \rangle$ $(\langle PaDS \rangle)\{\langle body \rangle\} \mid$	no local variables
	$\langle Ty \rangle$ $\langle Na \rangle$ $()\{\langle VaDS \rangle; \langle body \rangle\} \mid$	no parameters
	$\langle Ty \rangle$ $\langle Na \rangle$ $()\{\langle body \rangle\}$	only body
$\langle PaDS \rangle$	$\longrightarrow \langle VaD \rangle \mid \langle VaD \rangle, \langle PaDS \rangle$	parameter declaration sequence
$\langle body \rangle$	$\longrightarrow \langle rSt \rangle \mid \langle StS \rangle; \langle rSt \rangle$	function body

Figure 11: Original C0 grammar

A.2 Instruction Set Implementation

Translating MIPS mnemonics into 32-bit binary formats is not straightforward. Each instruction has its own operand layout, and the types and ranges of those operands all need to be validated before anything can be encoded. To keep this manageable, we structured the code around an abstract `Instruction` class, shown in Code 26.

```
public abstract class Instruction {
    protected final int opcode;

    protected Instruction(int opcode) {
        this.opcode = opcode;
    }

    public abstract int encode();

    public abstract boolean isResolved();

    public void execute(Configuration config) {
        // Default behavior is to throw an error for non-executable instructions.
        throw new UnsupportedOperationException("Cannot be executed.");
    }

    public String toBinaryString() {
        return Integer.toBinaryString(encode());
    }
}
```

Code 26: Abstract instruction class

Three concrete subclasses extend it, one for each MIPS format: **I-type**, **R-type**, and **J-type**. Each one handles the bit layout for its format, as shown in Code 27, Code 28, and Code 29. The J-type case needs a two-stage resolution process because the target address may not be known yet when the instruction is first encountered.

```
class ITypeInstruction extends Instruction {
    private final int rs, rt, immediate;

    public ITypeInstruction(int opcode, int rs, int rt, int immediate) {
        super(opcode);
        this.rs = rs;
        this.rt = rt;
        this.immediate = immediate;
    }

    @Override
    public int encode() {
        // opcode (6) | rs (5) | rt (5) | immediate (16)
        return (opcode << 26) | (rs << 21) | (rt << 16) | (immediate & 0xFFFF);
    }
    // ...
}
```

Code 27: I Type instruction class

```java
class RTypeInstruction extends Instruction {
    private final int rs, rt, rd, sa, fun;

    public RTypeInstruction(int opcode, int rs, int rt,
                            int rd, int sa, int fun) {
       // ... constructor ...
    }

    @Override
    public int encode() {
        // opcode (6) | rs (5) | rt (5) | rd (5) | sa (5) | fun (6)
        return (opcode << 26) | (rs << 21) | (rt << 16)
               (rd << 11) | (sa << 6) | fun;
    }
    // ...
}
```

Code 28: R Type instruction class

```java
public class JTypeInstruction extends Instruction {
    private final long address;
    private int iindex = -1;
    private boolean isResolved;

    // ... constructors and resolve() method ...

    @Override
    public int encode() {
        if (!isResolved) {
            throw new AssemblerException("Attempt to encode unresolved J-type.");
        }
        // opcode (6) | iindex (26)
        return (opcode << 26) | (iindex & 0x03FFFFFF);
    }
    // ...
}
```

Code 29: J Type instruction class

A.2.1 The Instruction Registry

Creating and managing instruction objects for both the assembler and the simulator is handled through a single centralized factory: the `InstructionRegistry` (Code 30). It sits between the high-level assembler and simulator logic on one side, and the details of individual instructions on the other.

```
public class InstructionRegistry {
    Map<String, InstructionCreator> encoderInstructionMap =
    Initializer.initializeEncoderMap();

    Map<String, InstructionCreator> executableInstructionMap =
    Initializer.initializeExecutableMap();

    Map<String, List<OperandType>> operandTypesMap =
    Initializer.initializeOperandTypesMap();

    public static Instruction createForEncoder(String mnemonic, int[] operands) {
        // ...
    }

    public static Instruction createForExecutor(String mnemonic, int[] operands) {
        // ...
    }
}
```

Code 30: Instruction registry

The registry uses a dual-map factory pattern. At startup, an `Initializer` class fills two separate maps:

- `encoderInstructionMap`: holds factory lambdas that produce basic `Instruction` objects for bytecode encoding.
- `executableInstructionMap`: holds factory lambdas that produce `Instruction` objects with simulation logic inside an overridden `execute` method.

```
class Initializer {
    // ... maps and initialization blocks ...

    private static void register(String mnemonic,
                                 List<OperandType> operandTypes,
                                 InstructionCreator encoder,
                                 InstructionCreator executor) {
        encoderInstructionMap.put(mnemonic, encoder);
        executableInstructionMap.put(mnemonic, executor);
        operandTypesMap.put(mnemonic, operandTypes);
    }

    private static void lw() {
        int opcode = 0b100011;
        register(
          "lw", // Mnemonic
          List.of(REG5, REG5, IMM16), // Operand types for parsing

          // Factory #1: For the Assembler/Encoder
          operands -> new ITypeInstruction(opcode, operands[1], operands[0],
operands[2]),

          // Factory #2: For the Simulator/Executor
          operands -> {
              int rs = operands[0], rt = operands[1], imm = operands[2];
              return new ITypeInstruction(opcode, rs, rt, imm) {
                  @Override
                  public void execute(Configuration c) {
                      int address = c.getRegister(rs) + imm;
                      int value = c.getWord(address);
                      c.setRegister(rt, value);
                      c.setPC(c.getPC() + 4);
                  }
              };
          });
    }
}
```

Code 31: Instruction registration

A.3 End-to-End Simulation

This section follows a single C0 program through the full pipeline, from source code to a simulated result. The example is a recursive function that computes `pow(2, 3)`, shown in Code 32. It was chosen because it exercises both the compiler's handling of recursion and the simulator's call stack management at the same time.

```
int res;
int pow(int a, int n){
  int res;
  if n==0 {
    res = 1
  }
  else {
    n = n-1;
    res = pow(a, n);
    res = res*a
  };

  return res
};

int main(){
  int a;
  int n;

  a = 2;
  n = 3;
  res = pow(a, n);

  return 1
}
```

Code 32: Example of a recursive C0 program.

A.3.1 Step 1: C0 to MIPS Assembly

The compiler takes the C0 source and produces MIPS assembly. This intermediate output mixes standard MIPS instructions with custom macros like `macro: gpr(...)` that the assembler expands later. The generated code, shown below, covers stack pointer initialization, the main function body, and the recursive `pow` function.

```
macro: gpr(30) = enc(10240, uint)
macro: gpr(28) = enc(5120, uint)
addiu 29 28 20
subu 1 29 28
srl 1 1 2
macro: zero(28, 1)
macro: gpr(28) = enc(5120, uint)
j _main

_main:
addi 1 29 -8
macro: gpr(2) = enc(2, int)
sw 2 1 0
addi 1 29 -4
macro: gpr(2) = enc(3, int)
sw 2 1 0
addi 1 28 0
addi 2 29 -8
lw 2 2 0
addi 3 29 -4
lw 3 3 0
addi 4 29 12
addi 4 4 -9216
blez 4 4
macro: gpr(1) = enc(41, uint)
sysc
addi 29 29 20
sw 1 29 -16
sw 2 29 -12
sw 3 29 -8
addi 1 29 -4
addi 2 0 1
macro: zero(1, 2)
jal _pow
macro: gpr(1) = enc(1, int)
sysc

_pow:
sw 31 29 -20
addi 1 29 -8
lw 1 1 0
macro: gpr(2) = enc(0, int)
bne 1 2 3
addi 1 0 1
blez 0 2
add 1 0 0
beq 1 0 6
addi 2 29 -4
macro: gpr(3) = enc(1, int)
sw 3 2 0
blez 0 46
addi 1 29 -8
addi 2 29 -8
lw 2 2 0
macro: gpr(3) = enc(1, int)
```

```
sub 2 2 3
sw 2 1 0
addi 1 29 -4
addi 2 29 -12
lw 2 2 0
addi 3 29 -8
lw 3 3 0
addi 4 29 12
addi 4 4 -9216
blez 4 4
macro: gpr(1) = enc(41, uint)
sysc
addi 29 29 20
sw 1 29 -16
sw 2 29 -12
sw 3 29 -8
addi 1 29 -4
addi 2 0 1
macro: zero(1, 2)
jal _pow
addi 1 29 -4
addi 2 29 -4
lw 2 2 0
addi 3 29 -12
lw 3 3 0
macro: mul(2, 2, 3)
sw 2 1 0
addi 1 29 -4
lw 1 1 0
lw 2 29 -16
sw 1 2 0
lw 1 29 -20
addi 29 29 -20
jr 1
```

A.3.2 Step 2: MIPS Assembly to Bytecode

The assembler takes that output, resolves labels, expands macros, and encodes each instruction into a 32-bit integer. The result is an array of bytecode ready to be loaded into the simulator's memory.

```
00111100000111100000000000000000
00110111110111100010100000000000
00111100000111000000000000000000
00110111100111000001010000000000
00100111100111010000000000010100
...
00100011011000100000000000000000
10101100001000100000000000000000
00100011101000011111111111111100
10001100001000010000000000000000
10001111101000101111111111110000
10101100010000010000000000000000
10001111101000011111111111101100
00100011101111011111111111101100
00000000001000000000000000001000
```

A.3.3 Step 3: Simulation and Final State

The bytecode gets loaded into a fresh `Configuration` and the simulator starts running. It works through each instruction, updating registers and memory as it goes. Once the program hits `syscall` and stops, the final configuration can be checked to see if the output is correct.

```
Configuration{
  pc = 184,
  gpr = {
    gpr 0 - 0
    gpr 1 - 1
    gpr 2 - 5120
    gpr 3 - 2
    gpr 4 - -4004
    gpr 5 - 0
    gpr 6 - 0
    ...
    gpr 25 - 0
    gpr 26 - 0
    gpr 27 - 8
    gpr 28 - 5120
    gpr 29 - 5140
    gpr 30 - 10240
    gpr 31 - 364
  }
}
```

Code 33: Final state of the simulator after execution.

The program stopped with the PC at address 184. Register `$v0` (`gpr[2]`) holds the memory address of the result, and reading that address with `config.getWord(5120)` gives `8`, which is the correct value for 2^3. Register `$a0` (`gpr[1]`) contains `1`, signaling a clean exit.

A.4 Program Tests

A suite of JUnit tests covers the full pipeline end-to-end. Each test compiles a C0 program to MIPS assembly, assembles it to bytecode, runs it through the simulator, and checks that the final register and memory values match what is expected. Examples are shown in Code 34.

```
class C0ProgramTest {
        private Configuration config;

        @BeforeEach
        void setUp() {
            config = new Configuration();
        }

        @Test
        void test_trivial_program() {
            // arrange
            String code = "int main(){asm( addi 2 0 3 ); return 1}~";
            C0Program pr = new C0Program(code);

            // act
            simulateProgram(pr);

            // assert
            assertEquals(1, config.getRegister(1));
            assertEquals(3, config.getRegister(2));
        }

        @Test
        void test_single_int_max_constant_program() {
            // arrange
            String code = "int a; int main(){a = 2147483647; return 1}~";
            C0Program pr = new C0Program(code);

            // act
            simulateProgram(pr);

            // assert
            int expectedValue = 2147483647;
            assertEquals(1, config.getRegister(1));
            assertEquals(expectedValue, config.getWord(pr.SBASE));
        }

        // 70 more tests
```

Code 34: Instruction Registration

A.5 Kernel C0-Program

This section provides the "C0 + Assembly" source code for simple operating system kernel, which was translated into *MIPS* bytecode.

The full *C0* implementation with comments for further explanation can be found in the project's `resources/c0 programs/AbstractKernel.c0` repository. The resulting *MIPS code* consists of just below 900 native instructions, which expands to over 1,600 instructions after *macro* expansion. It can be seen in `resources/mips programs`

A.6 Bootloader: Kernel Fetch Routine

Here, we describe *C0 + Assembly* program responsible for loading translated kernel code into *host* machine's predefined memory region. It can be seen with comments in `resources/c0 programs/FetchKernel.c0` repository

A.7 ROM Initialization

MIPS code provided below describes initial configuration of *host's* main memory - *ROM* region. It can be found in the project's repository: `resources/mips programs/ROM`.